Butterbear

Seven-minute presentation and tutorial script

Nemo and Aon each speak for 3:30. Use this guide for rehearsal; the detailed handoff explains the code and limitations. The tutorial video uses synthetic English narration, not either presenter's voice.

Presentation order and timing

Time	Lead	Screen / action
0:00–0:40	Nemo	Slide 1: Use Case Diagram
0:40–3:30	Nemo	Slide 2, then Runner core folder: required steps 1–12
3:30–4:10	Aon	Slide 3: Class Diagram
4:10–4:50	Aon	Slide 4: Sequence Diagram
4:50–6:35	Aon	Slide 5, extras folder, real Gmail inbox
6:35–7:00	Aon	Slide 6: work split and GitHub Contributors
7:00–10:00	Both	Questions

Rehearsal commands

Run from the project root. Credentials stay in the ignored local .env.

```
npm ci
npm run seed
npm start
# In a second terminal:
npm run demo -- --folder "1 Core steps 01–12"
# Show Class and Sequence Diagram slides.
npm run demo -- --folder "2 Extra credits step 15"
```

The core command saves generated demo state under ignored .local; extras resumes it. Running the complete `npm run demo` is useful for rehearsal, but does not pause for diagrams. Use only one demo client at a time.

Postman Runner option

Import `postman/Butterbear–Presentation.postman_collection.json`. Its display name is Butterbear Final Presentation. Run `npm run demo:env` to generate the ignored local environment; it contains demo credentials, not SMTP or Atlas passwords. Keep it local. Run folder 1 in one iteration, show the diagrams, then run folder 2 without clearing collection variables. Ensure Postman retains the variable changes from the first run; if the client does not retain them, use the tested Newman commands.

Before showing the screen

Open slides, the terminal/Runner, and the project inbox. Close the .env tab and hide unrelated email. The extras booking stays active until email inspection. Do not show tokens, database passwords, Gmail app passwords or exported secret environments.

Nemo: 0:00–3:30

0:00–0:40 / slide 1

“Butterbear is a campground booking API. A normal user can register, sign in, browse campgrounds, and manage their own bookings. An admin can manage any booking. Our additions are favorite campgrounds, special requests, and booking emails. A scheduled worker sends reminders with a forecast from Open-Meteo. We run Express locally and store the data in MongoDB Atlas.”

Point to the User and Admin actors, then the worker and external services. Use the diagram as the introduction; avoid an extra title slide.

0:40–1:20 / begin Runner core folder

“We demonstrate the core requirements in one ordered Runner workflow. First, registration with a missing phone is rejected. Valid registration creates a normal user. Login saves a token for later requests. The campground list provides the ID used by the booking requests.”

Start the whole core folder once. Explain results after completion, in the listed order. Expected error responses are successful validation checks.

1:20–2:10 / booking conditions and own operations

“A valid booking stays for three calendar nights. A four-night request is rejected. We use the campground’s time zone and a two PM check-in. The normal user can view, edit, and delete their own booking. The special request is stored with the booking. Delete marks it cancelled, and later reads no longer show it as active.”

Show the successful create and invalid-condition assertions. Explain that each booking belongs to the authenticated user; a client cannot choose another owner.

2:10–2:45 / logout

“Logout deletes the stored session for this token. The next profile request with that same token receives unauthorized. JWT signature checking alone would not provide immediate logout; the server also checks the database session.”

Show the expected 401 assertion. Keep normal-user and admin stages distinct when explaining the results.

2:45–3:30 / admin operations

“The admin logs in with a seeded account and can view, edit, and delete a booking belonging to the normal user. The collection creates a second booking for this stage, because the first one has already been deleted. These passing assertions show the full required user and admin lifecycle. Aon will now explain the data model and the asynchronous email flow.”

Do not manually issue each request. Return to slide 3 by 3:30, even if the Runner finished much earlier.

Aon: 3:30–7:00

3:30–4:10 / slide 3, Class Diagram

“User, Campground and Booking are the main models. A booking references one user and one campground. User stores favorite campground IDs. A session stores the token identifier and expiry so logout can revoke it. Notifications are embedded inside a booking, so a successful booking write also persists its required email work. Admin is a value of User.role, not a separate subclass.”

4:10–4:50 / slide 4, Sequence Diagram

“The API validates the session and dates, then saves the booking and email jobs together. It returns created with emailStatus queued. Separately, the worker claims a due job. For a reminder it fetches the check-in forecast, sends the email through Gmail, and records the result. Each edit increases the booking revision, so an old reminder can be recognized and skipped.”

4:50–5:25 / start extras folder

“Now we run the extra requirements together. A user adds, views, and removes a favorite. The collection fetches weather from Open-Meteo, creates a booking with a special request, and edits that request. It then waits for confirmation, update, and the latest reminder to complete. The request is optional, trimmed, and limited to two thousand characters.”

Start the extras folder. While it waits, explain that the usual reminder is due at 09:00 on the day before check-in. The demonstration uses a valid future booking whose reminder is already due; the implementation’s clock is not changed.

5:25–6:35 / show Gmail

“Here are the actual messages. The confirmation contains the initial booking details. The update contains the changed request. The reminder adds minimum and maximum temperature, rain probability, and the Open-Meteo source. If weather temporarily fails, we still send the reminder with a clear unavailable note. SMTP failures remain queued for retry. The backend must be running for the worker to send reminders.”

Show the three messages, then return to slide 5. The tutorial link opens the five-minute instructional video; do not play the entire video during the seven-minute presentation.

6:35–7:00 / slide 6, work split

“Nemo owns the review packages for authentication, favorites, and booking creation. I own booking management, email delivery, and weather reminders. We split speaking time equally. The repository link and GitHub Contributors show the actual commit history. The implementation was AI assisted; we each need to understand and maintain the code we present.”

Open Contributors only if the real history is ready. Aon’s username and commits have not been fabricated. Stop at 7:00.

Questions and submission checklist

Short answers to likely questions

Why MongoDB Atlas? It matches the course database constraint and persists bookings and jobs independently of the local Node process.
Why embedded notification jobs? The booking and initial jobs are written atomically in the same document, without a separate queue broker.
Exactly-once email? No. A crash after SMTP acceptance can cause a retry and duplicate. Claim tokens and leases reduce concurrent processing; they do not guarantee exactly once.
Two users book the same dates? Allowed. The brief has no pitch inventory or campground capacity model.
What is the external API? Open-Meteo. Gmail is the delivery channel; weather is fetched and included in the reminder.
Why can a reminder run now? Its normal scheduled time has already passed, while check-in remains in the future. This is normal late-booking behavior.
What if weather fails? Public weather returns 503; a reminder sends a clear fallback note. Retry the forecast before presenting a successful weather example.
What protects another user's booking? Owner filters on read, edit, delete, and notification routes. Admin uses the stored role to access any booking.

Actual team work still to record

The split is a balanced assignment of review and maintenance packages, not invented historical authorship. Aon should study his files, make genuine changes or fixes, and commit using his own identity. Nemo should do the same for his packages. Add the final repository link to the slides if the team moves to a new repository. Never backdate or impersonate commits.

Files to submit / share

- Main presentation PowerPoint and PDF; repository and tutorial links are included.
- Detailed handoff LaTeX/PDF for studying the source and limitations.
- This presenter guide, plus the five-minute tutorial MP4 and editable source slides.
- Source code and collection pushed to GitHub. .env and .local must remain excluded.

The tutorial is an instructional walkthrough of the actual implementation with synthetic English narration. Check the course's authorship/narration policy before using it as the final extra-credit submission. The transcript below can instead be recorded in the presenters' own voices without changing the teaching sequence.

For the Tuesday presentation on 17 November 2026, the stated deadline is midnight at the end of Monday 16 November, subject to the group's assigned schedule. The Sunday section has a different deadline. Upload the required presentation to MCV, verify the attachment opens, and check the assigned slot. No MCV submission has been performed by this workflow.

Five-minute tutorial transcript: 0:00–2:30

0:00–0:30 / Build Butterbear extras

This tutorial explains how the Butterbear extra requirements are implemented. The project uses Express, Mongoose, MongoDB Atlas, and Nodemailer. The first half covers the data model and routes assigned to Nemo for review. The second half covers email delivery and weather assigned to Aon. These assignments describe learning and maintenance responsibilities. The implementation is AI assisted, and this video uses synthetic narration.

0:30–1:00 / Store favorites on the user

Start with the User schema. Favorites is an array of campground references. After authenticating the user, the add route checks that the campground is active and uses MongoDB add to set. This prevents duplicate favorites, including concurrent additions. The remove route uses pull. The list route populates active campground references and returns only the signed in user’s list. Demo the add, list, and remove requests together.

1:00–1:30 / Save a special request

Next, add specialRequest to the Booking schema. It defaults to an empty string and has a limit of two thousand characters. The shared text validator checks the type and length, then trims whitespace. Creation stores the value, and editing preserves the old request when the field is omitted. The request also enters the notification snapshot so each email describes the version of the booking that caused it.

1:30–2:00 / Validate the booking dates

Use calendar dates in the campground’s time zone instead of counting fixed twenty four hour periods. The booking validator permits one through three nights and rejects a check in whose two PM arrival time has passed. It calculates the reminder for nine AM on the previous day. If that reminder time has already passed but check in remains in the future, the job is due now. This supports a genuine live demonstration.

2:00–2:30 / Persist the booking and email jobs

Create the booking and its initial notification jobs in a single MongoDB document write. The helper builds a confirmation due immediately and a reminder with its scheduled time. Each job stores the recipient, revision, and a snapshot of booking details. Return HTTP two hundred and one with emailStatus queued after persistence. Do not wait for SMTP inside the request. This keeps a slow email provider from blocking the booking response.

Five-minute tutorial transcript: 2:30–5:00

2:30–3:00 / Handle edits and admin access

Aon's section starts with booking edits. Normal users receive an owner filter, while an admin can manage any user's booking. Revalidate the new dates and special request. The update matches the current revision and increments it while appending an update email and replacement reminder. A concurrent change returns a conflict instead of silently overwriting another edit. The email recipient remains the booking owner, even when an admin performs the edit.

3:00–3:30 / Claim and deliver notification jobs

The worker polls every five seconds and atomically claims a due pending job. A claim token and lease identify the worker processing it. Before sending, it checks that the booking is still active and that a reminder belongs to the latest revision. A successful SMTP call records sent status. Failures retry with increasing delays and stop after five attempts. Delivery is at least once, so a crash after SMTP acceptance can produce a duplicate.

3:30–4:00 / Fetch weather for the check-in date

For reminders, call Open Meteo using the campground coordinates, time zone, and check in date. Request daily temperature, rain probability, and weather code. The service verifies the returned date and numeric values, then caches a valid forecast for fifteen minutes. The provider supports dates from today through fifteen days ahead. If weather is unavailable, the worker sends a clear unavailable note instead of losing the booking reminder. The external API is weather, not email.

4:00–4:30 / Send real email through Gmail

Configure Nodemailer for Gmail SMTP on port five hundred eighty seven with STARTTLS. Keep the Gmail app password and MongoDB connection string in the ignored environment file. Never include them in the repository, slides, or recording. Startup verifies the SMTP connection. Each plain text email includes the booking ID, campground, dates, and special request. The reminder adds temperature, rain probability, and Open Meteo attribution. A sent state records SMTP acceptance; inspect the real inbox for receipt.

4:30–5:00 / Demonstrate and explain the result

Finish by running the ordered presentation collection. The core folder proves registration, login, campground viewing, booking rules, user operations, logout, and admin operations. Show the class and sequence diagrams before running the extras folder. Then inspect the real Gmail inbox for confirmation, update, and the weather reminder. Explain that the backend must remain running for scheduled work. Both teammates should study their assigned source files, rehearse the seven minute presentation, and describe only contributions they actually made.